# FAST National University of Computer and Emerging Sciences

Department of Data Science


# Artificial Intelligence

## Assignment #2

*Local Search Algorithms & Genetic Algorithms*


**Name:** Ali Ahmad

**Roll No.:** 23L-2619

**Section:** BSDS-6A

**Instructor:** Ms. Hajra Waheed


**Date:** April 12, 2026

**GitHub:** https://github.com/whozahm3d/AI-Assignment-2

| **Section** | **Marks** |
|---|---|
| Section A: Local Search Algorithms | / 40 |
| Section B: Genetic Algorithms | / 35 |
| Section C: Real-World Application | / 25 |
| **Total** | / 100 |

Submitted in partial fulfilment of the requirements for
Artificial Intelligence, Spring 2026

# Contents


**1 Section A: Local Search Algorithms [40 Marks] 1**

1.1 Q1 — First-Choice & Stochastic Hill Climbing [14 Marks] . . . . . 1

1.2 Q2 — Random Restart Hill Climbing [14 Marks] . . . . . 5

1.3 Q3 — Diagnosing & Fixing HC Failures [12 Marks] . . . . . 8

**2 Section B: Genetic Algorithms [35 Marks] 11**

2.1 Q4 — Full Genetic Algorithm from Scratch [20 Marks] . . . . . 11

2.2 Q5 — GA for the Course Scheduling Problem [15 Marks] . . . . . 15

**3 Section C: Real-World Application [25 Marks] 18**

3.1 Q6 — Driver Positioning System [25 Marks] . . . . . 18

# 1 Section A: Local Search Algorithms [40 Marks]

## 1.1 Q1 — First-Choice & Stochastic Hill Climbing [14 Marks]

### Problem Setup

The Q1 landscape is defined over states $s \in \{1, \ldots, 12\}$ where neighbours are the immediate left and right states only. Global maximum: state 11, $f(11) = 16$.

| **State** | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $f(s)$ | 4 | 9 | 6 | 11 | 8 | 15 | 10 | 7 | 13 | 5 | 16 | 12 |

### Code

```python
import random

LANDSCAPE = [4, 9, 6, 11, 8, 15, 10, 7, 13, 5, 16, 12]
#             s1  s2  s3  s4  s5  s6  s7  s8  s9 s10 s11 s12

def get_neighbours_q1(state, landscape):
    idx = state - 1
    neighbours = []
    if idx - 1 >= 0:
        neighbours.append(state - 1)
    if idx + 1 < len(landscape):
        neighbours.append(state + 1)
    return neighbours
```

Listing 1: Landscape and neighbour helper

```python
def first_choice_hc(landscape, start):
    current = start
    path = [current]
    while True:
        neighbours = get_neighbours_q1(current, landscape)
        moved = False
        for neighbour in neighbours:
            if landscape[neighbour - 1] > landscape[current - 1]:
                current = neighbour
                path.append(current)
                moved = True
                break
        if not moved:
            break
    return path, current
```

Listing 2: First-Choice Hill Climbing

```python
def stochastic_hc(landscape, start):
    current = start
    path = [current]
    while True:
        neighbours = get_neighbours_q1(current, landscape)
        uphill = [n for n in neighbours
                  if landscape[n - 1] > landscape[current - 1]]
        if len(uphill) == 0:
```

```
            break
        current = random.choice(uphill)
        path.append(current)
    return path, current
```

Listing 3: Stochastic Hill Climbing

## Part (a) — All 12 Starting States [5 Marks]

```
PART (a) - Running both algorithms from all 12 starting states
Landscape: [4, 9, 6, 11, 8, 15, 10, 7, 13, 5, 16, 12]

Start   Algorithm              Path                      Terminal
   Steps
-------------------------------------------------------------------------

1       First-Choice HC        [1, 2]                    2          1
1       Stochastic HC          [1, 2]                    2          1
2       First-Choice HC        [2]                       2          0
2       Stochastic HC          [2]                       2          0
3       First-Choice HC        [3, 2]                    2          1
3       Stochastic HC          [3, 4]                    4          1
4       First-Choice HC        [4]                       4          0
4       Stochastic HC          [4]                       4          0
5       First-Choice HC        [5, 4]                    4          1
5       Stochastic HC          [5, 6]                    6          1
6       First-Choice HC        [6]                       6          0
6       Stochastic HC          [6]                       6          0
7       First-Choice HC        [7, 6]                    6          1
7       Stochastic HC          [7, 6]                    6          1
8       First-Choice HC        [8, 7, 6]                 6          2
8       Stochastic HC          [8, 9]                    9          1
9       First-Choice HC        [9]                       9          0
9       Stochastic HC          [9]                       9          0
10      First-Choice HC        [10, 9]                   9          1
10      Stochastic HC          [10, 11]                  11         1
11      First-Choice HC        [11]                      11         0
11      Stochastic HC          [11]                      11         0
12      First-Choice HC        [12, 11]                  11         1
12      Stochastic HC          [12, 11]                  11         1
```

## Part (b) — Divergence Analysis and 50-Trial Test [4 Marks]

```
[b.1] Global maximum (state 11, f=16) summary table

Start   FC Terminal   FC->11?   ST Terminal   ST->11?
-----------------------------------------------------
1       2             no        2             no
2       2             no        2             no
3       2             no        4             no
4       4             no        4             no
5       4             no        6             no
6       6             no        6             no
7       6             no        6             no
8       6             no        9             no
9       9             no        9             no
10      9             no        11            YES
```

```
11       11            YES        11             YES
12       11            YES        11             YES

Summary:  First-Choice HC reaches state 11 from  2/12 starting states
          Stochastic HC  reaches state 11 from  3/12 starting states

[b.2] States where FC and Stochastic reach DIFFERENT terminals

Start=3:  FC: [3,2]->s2(f=9)  | ST: [3,4]->s4(f=11)
  At s3(f=6): Left=s2(f=9) <- FC picks left; Right=s4(f=11)
  FC moves to s2 and gets trapped. Stochastic randomly chose right.

Start=5:  FC: [5,4]->s4(f=11) | ST: [5,6]->s6(f=15)
  At s5(f=8): Left=s4(f=11) <- FC picks left; Right=s6(f=15)
  FC gets trapped at s4. Stochastic chose right -> s6.

Start=8:  FC: [8,7,6]->s6(f=15) | ST: [8,9]->s9(f=13)
  At s8(f=7): Left=s7(f=10) <- FC picks left; Right=s9(f=13)
  FC reaches s6 via s7. Stochastic chose right -> s9.

Start=10: FC: [10,9]->s9(f=13) | ST: [10,11]->s11(f=16) [GLOBAL MAX]
  At s10(f=5): Left=s9(f=13) <- FC picks left; Right=s11(f=16)
  FC misses the global max by one step. Stochastic chose right -> s11
      .

[b.3] Stochastic HC run 50 times from start=4 (no fixed seed)
  Reached state 11 (global max): 0/50 times = 0.0%
  All terminals: {4: 50}
```

**Explanation (b.1):** The landscape has five local maxima: states 2 ($f = 9$), 4 ($f = 11$), 6 ($f = 15$), 9 ($f = 13$), and 11 ($f = 16$). First-Choice always takes the left neighbour first, causing it to be trapped at earlier local maxima. Stochastic HC picks randomly among all uphill moves and may therefore reach a different local maximum from the same start.

**Explanation (b.2):** Divergence happens at states 3, 5, 8, and 10 because at each of these states both neighbours are strictly uphill, but First-Choice always picks the left one while Stochastic selects randomly. These are the only four states with two simultaneous uphill directions.

**Explanation (b.3):** From state 4 ($f = 11$), the left neighbour $s_3$ ($f = 6$) and right neighbour $s_5$ ($f = 8$) are both strictly worse, so state 4 is itself a local maximum. Every HC run terminates immediately regardless of randomness, giving a 0% success rate. This reveals that Stochastic HC's reliability depends entirely on the basin structure: it is completely unreliable when the starting state is already a local trap.

### Part (c) — Plateau Handling [5 Marks]

Modified landscape: states $\{5, 6, 7\}$ all set to $f = 15$.

```
LANDSCAPE_PLATEAU = [4, 9, 6, 11, 15, 15, 15, 7, 13, 5, 16, 12]

def first_choice_hc_plateau(landscape, start):
```

```python
    current = start; path = [current]; plateau_hit = False
    while True:
        neighbours = get_neighbours_q1(current, landscape)
        moved = False
        for nb in neighbours:
            if landscape[nb - 1] > landscape[current - 1]:
                current = nb; path.append(current); moved = True; break
        if not moved:
            equal = [nb for nb in neighbours
                     if landscape[nb - 1] == landscape[current - 1]]
            if equal and not plateau_hit:
                print(f"[WARNING] Plateau detected at state {current}"
                      f"(f={landscape[current-1]}). No strictly better "
                      f"neighbour.")
                plateau_hit = True
            break
    return path, current, plateau_hit

def first_choice_hc_sideways(landscape, start, max_sideways=10):
    current = start; path = [current]; sideways_count = 0
    while True:
        neighbours = get_neighbours_q1(current, landscape)
        moved = False
        for nb in neighbours:
            if landscape[nb - 1] > landscape[current - 1]:
                current = nb; path.append(current); moved = True; break
        if not moved:
            equal = [nb for nb in neighbours
                     if landscape[nb - 1] == landscape[current - 1]]
            if equal and sideways_count < max_sideways:
                current = equal[0]; path.append(current)
                sideways_count += 1; moved = True
        if not moved:
            break
    return path, current
```

Listing 4: Plateau-aware HC with sideways-move extension

```
[c.1] WITHOUT sideways moves

Start=1:  path=[1,2],     terminal=2  (f=9),  plateau=no
Start=2:  path=[2],       terminal=2  (f=9),  plateau=no
Start=3:  path=[3,2],     terminal=2  (f=9),  plateau=no
[WARNING] Plateau detected at state 5 (f=15). No strictly better neighbour.
Start=4:  path=[4,5],     terminal=5  (f=15), plateau=YES
[WARNING] Plateau detected at state 5 (f=15). No strictly better neighbour.
Start=5:  path=[5],       terminal=5  (f=15), plateau=YES
[WARNING] Plateau detected at state 6 (f=15). No strictly better neighbour.
Start=6:  path=[6],       terminal=6  (f=15), plateau=YES
[WARNING] Plateau detected at state 7 (f=15). No strictly better neighbour.
```

```
Start=7:  path=[7],       terminal=7  (f=15), plateau=YES
[WARNING] Plateau detected at state 7 (f=15). No strictly better
   neighbour.
Start=8:  path=[8,7],     terminal=7  (f=15), plateau=YES
Start=9:  path=[9],       terminal=9  (f=13), plateau=no
Start=10: path=[10,9],    terminal=9  (f=13), plateau=no
Start=11: path=[11],      terminal=11 (f=16), plateau=no
Start=12: path=[12,11],   terminal=11 (f=16), plateau=no

=> First-Choice HC stuck on plateau in 5/12 runs
   Stochastic HC stuck on plateau in 5/12 runs

[c.2] WITH sideways moves (cap=10)

Start=4: path=[4,5,6,7,6,7,6,7,6,7,6,7], terminal=7 (f=15),
   reached_11=no
...
=> First-Choice HC with sideways: reached global max in 2/12 runs
=> Stochastic HC with sideways:   reached global max in 3/12 runs

Summary:
  Without sideways: FC stuck 5/12, Stochastic stuck 5/12
  With sideways:    FC global max 2/12, Stochastic global max 3/12
```

**Analysis (without sideways):** When states 5, 6, 7 all have $f = 15$, any algorithm reaching one of them sees only equal-valued or lower neighbours, so the strict uphill condition never fires. Without sideways moves the algorithm stops at the plateau edge, unable to cross it and reach state 11 ($f = 16$).

**Analysis (with sideways moves):** The extension does not work equally well for both variants. First-Choice HC's deterministic left-first bias causes oscillation between adjacent plateau states ($s_6 \leftrightarrow s_7$), exhausting the cap of 10 without net rightward progress. Stochastic HC benefits marginally more (3/12 vs. 2/12) because random choice among equal neighbours occasionally drifts rightward toward $s_8$. The cap of 10 is critical: without it the algorithm loops indefinitely on the plateau. Stochastic HC is the clearer beneficiary because randomness occasionally produces net rightward drift, whereas First-Choice wastes every sideways step on pure oscillation.

## 1.2 Q2 — Random Restart Hill Climbing [14 Marks]

### Problem Setup

Warehouse robot across 14 positions. Global maximum: state 11, $f(11) = 19$.

| **State** | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $f(s)$ | 5 | 8 | 6 | 12 | 9 | 7 | 17 | 14 | 10 | 6 | 19 | 15 | 11 | 8 |

```python
def find_local_maxima(landscape):
    local_maxima = []
    n = len(landscape)
    for i in range(n):
        val   = landscape[i]
        left  = landscape[i-1] if i > 0     else -9999
```

```python
        right = landscape[i+1] if i < n - 1 else -9999
        if val > left and val > right:
            local_maxima.append(i + 1)
    return local_maxima

def random_restart_hc(landscape, num_restarts, variant='first_choice'):
    best_state, best_value = None, -999999
    all_results = []
    for _ in range(num_restarts):
        start = random.randint(0, len(landscape) - 1) + 1
        if variant == 'first_choice':
            path, terminal = first_choice_hc(landscape, start)
        else:
            path, terminal = stochastic_hc(landscape, start)
        val = landscape[terminal - 1]
        all_results.append((start, terminal, path))
        if val > best_value:
            best_value = val; best_state = terminal
    return best_state, best_value, all_results
```

Listing 5: RRHC implementation

## Part (a) — Local Maxima + RRHC Logs [4 Marks]

```
[a.1] Local Maxima in Q2 Landscape:
  State  2 -> f(2)  =  8
  State  4 -> f(4)  = 12
  State  7 -> f(7)  = 17
  State 11 -> f(11) = 19
  Total local maxima found: 4

[a.2] RRHC num_restarts=20 -- First-Choice variant (seed=100)
Restart   Start   Terminal   f-value   Global Max?
--------------------------------------------------
1         3       2          8         no
2         8       7          17        no
3         8       7          17        no
4         13      11         19        YES
5         3       2          8         no
6         12      11         19        YES
7         7       7          17        no
8         12      11         19        YES
9         6       4          12        no
10        7       7          17        no
11        9       7          17        no
12        13      11         19        YES
13        2       2          8         no
14        9       7          17        no
15        2       2          8         no
16        2       2          8         no
17        12      11         19        YES
18        8       7          17        no
19        5       4          12        no
20        1       2          8         no

Best state: 11 | Best f-value: 19 | Global max: YES
```

```
[a.3] RRHC num_restarts=20 -- Stochastic variant (seed=200)
(Global max found in multiple restarts; Best: 19 at state 11)
```

**Explanation:** A local maximum is any state whose $f$-value strictly exceeds both its neighbours. This landscape has four local peaks (states 2, 4, 7, 11), which is why basic HC gets trapped. Random Restart solves this by re-running HC from a fresh random start; over many restarts the algorithm is statistically likely to start in the basin of attraction of state 11 and climb directly to it.

### Part (b) — Empirical vs. Theoretical Probability [5 Marks]

Theoretical formula: $P(n) = 1 - (1-p)^n$, where $p = \frac{4}{14} \approx 0.2857$.

```
[b.1] States reaching global max (state 11) via First-Choice HC:
  States 11, 12, 13, 14 reach s11 -> 4/14 starts
  p = 4/14 = 0.2857

[b.2] Empirical Probability (100 independent trials per n)
  n    Successes/100   Empirical P
  1    31              0.3100
  3    56              0.5600
  5    83              0.8300
  10   97              0.9700
  20   99              0.9900

[b.3] Theoretical Probability P = 1-(1-p)^n  [p=0.2857]
  n    Calculation                Theoretical P
  1    1-(1-0.2857)^1             0.2857
  3    1-(1-0.2857)^3             0.6356
  5    1-(1-0.2857)^5             0.8141
  10   1-(1-0.2857)^10            0.9654
  20   1-(1-0.2857)^20            0.9988

[b.4] Empirical vs Theoretical
  n    Empirical P   Theoretical P   Difference
  1    0.3100        0.2857          +0.0243
  3    0.5600        0.6356          -0.0756
  5    0.8300        0.8141          +0.0159
  10   0.9700        0.9654          +0.0046
  20   0.9900        0.9988          -0.0088
```

**Analysis:** The empirical and theoretical probabilities align closely for small $n$, validating the independence assumption. The largest divergence at $n = 3$ (empirical 0.56 vs. theoretical 0.64) suggests sequential restarts may not be perfectly independent in finite trials; chance clustering in low-$p$ basins can occur. The formula provides a useful upper bound but slightly overestimates success probability at mid-range restart counts.

### Part (c) — Plateau Investigation [5 Marks]

```
Modified: states 7 and 8 both set to f=17
Original landscape: [5, 8, 6, 12, 9,  7, 17, 14, 10, 6, 19, 15, 11,
    8]
Modified landscape: [5, 8, 6, 12, 9,  7, 17, 17, 10, 6, 19, 15, 11,
    8]
```

```

[c.1] BEFORE plateau: Global max discovery rate: 50/50 = 100.00%

[c.2] AFTER plateau (s7=s8=17), 20 restarts:
  Average plateau terminations per run: 5.9/20 restarts

Summary:
  Original (no plateau):   100.00% discovery rate, ~0.0 plateau-stuck
  With plateau (s7=s8=17): 100.00% discovery rate, ~5.9 plateau-stuck
```

**Analysis:** Even with the plateau modification, the global max discovery rate stays at 100% across 50 independent trials because enough random starts land outside the plateau basin and climb directly to state 11. However, on average 5.9 of every 20 restarts ($\approx 30\%$) terminate on the plateau at $f = 17$, wasting computational effort. Running more restarts does eventually overcome the plateau — the experimental numbers confirm that 100% discovery is maintained — but at the cost of proportionally more wasted evaluations per trial. The plateau reduces effective restart efficiency rather than blocking discovery outright.

## 1.3 Q3 — Diagnosing & Fixing HC Failures [12 Marks]

### Part (a) — The diagnose_hc Function [4 Marks]

```python
def diagnose_hc(landscape, start):
    current = start; path = [current]; visited = set([current])
    while True:
        neighbours = get_neighbours_q1(current, landscape)
        moved = False
        for nb in neighbours:
            if landscape[nb - 1] > landscape[current - 1]:
                current = nb; path.append(current)
                if current in visited:
                    print(f"Terminated at {current}. Failure: RIDGE")
                    return path, current, "RIDGE"
                visited.add(current); moved = True; break
        if not moved:
            equal = [nb for nb in neighbours
                     if landscape[nb - 1] == landscape[current - 1]]
            mode = "PLATEAU" if equal else "LOCAL_MAX"
            print(f"Terminated at {current} with f={landscape[current
                -1]}."
                  f" Failure: {mode}")
            return path, current, mode
```

Listing 6: diagnose_hc: automatic failure-mode detection

```
--- Failure Mode 1: LOCAL MAXIMUM ---
Landscape: [3, 5, 2, 4, 1]
Start=1. Right s2(f=5) is better -> moves to s2.
At s2(f=5): left=s1(f=3) worse, right=s3(f=2) worse -> stuck.
Terminated at state 2 with f=5. Failure: LOCAL_MAX
Path: [1, 2]  Mode: LOCAL_MAX

--- Failure Mode 2: PLATEAU ---
Landscape: [4, 8, 12, 12, 12, 9]
```

```
States 3,4,5 form a flat plateau at f=12.
Start=2. Right s3(f=12) is better -> moves to s3.
At s3(f=12): right=s4(f=12) EQUAL -> PLATEAU detected.
Terminated at state 3 with f=12. Failure: PLATEAU
Path: [2, 3]  Mode: PLATEAU

--- Failure Mode 3: RIDGE ---
Landscape: [2, 5, 8, 6, 9, 4, 2]
Two peaks: s3(f=8) and s5(f=9) separated by valley s4(f=6).
Start=4. Left s3(f=8) better -> moves to s3.
At s3: all neighbours worse -> stuck at s3.
Reaching s5(f=9) requires going DOWNHILL through s4 -- forbidden.
Terminated at state 3 with f=8. Failure: LOCAL_MAX
  (Ridge confirmed: a higher unreachable peak exists across the valley.)
Path: [4, 3]  Mode: LOCAL_MAX
```

**Why HC cannot recover from a Local Maximum:** At a local maximum every adjacent state has a strictly lower $f$-value. HC has no mechanism to move downhill; only random restarts or a downhill-tolerating algorithm such as Simulated Annealing can escape.

**Why HC cannot recover from a Plateau:** On a plateau the strict uphill condition is never satisfied because all reachable neighbours have the same $f$-value. The algorithm stops at the first plateau state even if better regions lie beyond; sideways moves with a cap are the standard fix.

**Why HC cannot recover from a Ridge:** A ridge occurs when the global maximum sits across a valley from the current local peak. The only route to the higher peak requires moving downhill through the valley, which HC strictly forbids. Random restarts are the standard escape: a new start may land on the correct side of the valley.

### Part (b) — N-Queens ($N = 8$): Stochastic HC + RRHC [4 Marks]

```python
def count_conflicts(board):
    n = len(board); conflicts = 0
    for i in range(n):
        for j in range(i + 1, n):
            if board[i] == board[j]:                    conflicts += 1
            if abs(board[i]-board[j]) == abs(i-j):      conflicts += 1
    return conflicts

def stochastic_hc_nqueens(board):
    n = len(board); current = list(board)
    cur_conf = count_conflicts(current)
    while True:
        best_moves = []
        for col in range(n):
            for row in range(n):
                if row == current[col]: continue
                candidate = list(current); candidate[col] = row
                c = count_conflicts(candidate)
```

```python
                if c < cur_conf: best_moves.append(candidate)
        if not best_moves: break
        current = random.choice(best_moves)
        cur_conf = count_conflicts(current)
    return current, cur_conf

def solve_nqueens_rrhc(num_restarts):
    n = 8
    for attempt in range(num_restarts):
        board = [random.randint(0, n-1) for _ in range(n)]
        result, conf = stochastic_hc_nqueens(board)
        if conf == 0: return result, conf, attempt + 1
    return None, -1, num_restarts
```

Listing 7: N-Queens conflict count, stochastic HC, RRHC solver

```
[b.1] count_conflicts on known boards:
  [0,1,2,3,4,5,6,7] -> conflicts = 28  (all on main diagonal)
  [0,4,7,5,2,6,1,3] -> conflicts = 0   <- SOLUTION
  [3,6,2,7,1,4,0,5] -> conflicts = 0   <- SOLUTION
  [0,0,0,0,0,0,0,0] -> conflicts = 28  (all in row 0)

[b.2] solve_nqueens_rrhc(100):
  Restarts needed: 9
  Final conflicts: 0
  Final board:     [4, 0, 7, 5, 2, 6, 1, 3]
  Result: SOLUTION FOUND -- 0 conflicts!

  Visual board (Q = queen):
  +---+---+---+---+---+---+---+---+
  | . | Q | . | . | . | . | . | . |
  +---+---+---+---+---+---+---+---+
  | . | . | . | . | . | . | Q | . |
  +---+---+---+---+---+---+---+---+
  | . | . | . | . | Q | . | . | . |
  +---+---+---+---+---+---+---+---+
  | . | . | . | . | . | . | . | Q |
  +---+---+---+---+---+---+---+---+
  | Q | . | . | . | . | . | . | . |
  +---+---+---+---+---+---+---+---+
  | . | . | . | Q | . | . | . | . |
  +---+---+---+---+---+---+---+---+
  | . | . | . | . | . | Q | . | . |
  +---+---+---+---+---+---+---+---+
  | . | . | Q | . | . | . | . | . |
  +---+---+---+---+---+---+---+---+
```

**Explanation:** Each column holds exactly one queen. The column-based neighbourhood ($N \times N = 64$ moves) evaluates every possible row placement for every column at each step, dramatically reducing local minima compared to pairwise-swap approaches. Stochastic selection avoids the deterministic bias of First-Choice, exploring more diverse conflict-reduction paths. Random restarts escape local minima by generating a completely fresh random board whenever no improving move exists.

### Part (c) — Benchmark: 30 Trials per $k$ [4 Marks]

```
PART (c) - Benchmarking solve_nqueens_rrhc(k) [N=8, 30 trials each]

k       Successes/30    Success Rate    Avg Restarts to Solution
5       17/30           56.67%          2.5
10      25/30           83.33%          4.2
25      28/30           93.33%          7.3
50      30/30           100.00%         7.6
100     30/30           100.00%         6.7
```

**Analysis:** Two structural reasons explain why Stochastic HC struggles with N-Queens. **Reason 1 — Deceptive near-solution geometry:** There are only 92 solutions out of $8^8 = 16{,}777{,}216$ boards (density $\approx 0.0005\%$). When conflicts drop to 1–2 attacking pairs, almost all single-queen moves that remove the last conflict simultaneously introduce new diagonal attacks. **Reason 2 — Vanishing improving neighbourhood:** As conflicts decrease toward 1, the fraction of improving moves out of 64 column-row combinations shrinks dramatically. The benchmark confirms both: success rate rises from 56.67% at $k = 5$ to 100% at $k = 50$. For $N = 1000$, RRHC cannot scale because each board has $10^6$ column-row combinations to evaluate per HC step; the **MIN-CONFLICTS** algorithm solves $N = 1{,}000{,}000$ in near-linear time by directly targeting the most-conflicted queen.

# 2 Section B: Genetic Algorithms [35 Marks]

## 2.1 Q4 — Full Genetic Algorithm from Scratch [20 Marks]

### Problem Setup

Maximise $f(x) = -x^2 + 14x + 5$ for $x \in \{0, \ldots, 15\}$ using a 4-bit binary chromosome (MSB first). Global maximum: $x = 7$, $f(7) = 54$.

### Part (a) — Core GA Components [5 Marks]

```python
import random

def decode(chromosome):
    x = 0
    for bit in chromosome:
        x = x * 2 + bit
    return x

def fitness(chromosome):
    x = decode(chromosome)
    return -(x * x) + 14 * x + 5

def roulette_select(population):
    raw_scores = [fitness(ind) for ind in population]
    min_score  = min(raw_scores)
    offset     = abs(min_score) + 1 if min_score <= 0 else 0
    adjusted   = [s + offset for s in raw_scores]
    total      = sum(adjusted)
    spin = random.random(); cumulative = 0.0
    for i in range(len(population)):
        cumulative += adjusted[i] / total
```

```python
        if spin <= cumulative:
            return population[i]
    return population[-1]

def single_point_crossover(parent1, parent2, point):
    offspring1 = parent1[:point] + parent2[point:]
    offspring2 = parent2[:point] + parent1[point:]
    return offspring1, offspring2

def mutate(chromosome, p_m):
    return [1 - bit if random.random() < p_m else bit
            for bit in chromosome]
```

Listing 8: decode, fitness, roulette_select, crossover, mutate

```
--- TEST 1: decode(chromosome) ---
  Chromosome        Working                    x
  [0, 1, 1, 0]     0*8+1*4+1*2+0*1 = 6      6  OK
  [1, 0, 0, 1]     1*8+0*4+0*2+1*1 = 9      9  OK
  [1, 1, 0, 0]     1*8+1*4+0*2+0*1 = 12    12  OK
  [0, 0, 1, 1]     0*8+0*4+1*2+1*1 = 3      3  OK

--- TEST 2: fitness(chromosome) [f(x) = -x^2 + 14x + 5] ---
  Chromosome    x     Working              f(x)
  [0,1,1,0]     6     -36 + 84 + 5         53
  [1,0,0,1]     9     -81 + 126 + 5        50
  [1,1,0,0]     12    -144 + 168 + 5       29
  [0,0,1,1]     3     -9  + 42  + 5        38
  Reference: x=7 -> f(7) = -49+98+5 = 54  [GLOBAL MAXIMUM]

--- TEST 3: roulette_select ---
  Total fitness = 170
  Individual    f(x)   Prob = f/total   Cumulative
  [0,1,1,0]    53      0.3118           0.3118
  [1,0,0,1]    50      0.2941           0.6059
  [1,1,0,0]    29      0.1706           0.7765
  [0,0,1,1]    38      0.2235           1.0000
  Empirical check (10,000 spins): probabilities match theory.

--- TEST 4: single_point_crossover ---
  Parent1=[0,1,1,0] (x=6,f=53)  Parent2=[1,0,0,1] (x=9,f=50)
  Point   Offspring1      x   f      Offspring2      x    f
  1       [0, 0, 0, 1]    1   18     [1, 1, 1, 0]   14    5
  2       [0, 1, 0, 1]    5   50     [1, 0, 1, 0]   10   45
  3       [0, 1, 1, 1]    7   54  <- GLOBAL MAX!
          [1, 0, 0, 0]    8   53

--- TEST 5: mutate ---
  [0,1,1,0] p_m=0.0 -> [0,1,1,0]  (unchanged)
  [0,1,1,0] p_m=1.0 -> [1,0,0,1]  (all bits flipped)
```

## Part (b) — The run_ga Function [5 Marks]

```python
def run_ga(pop_size, num_generations, p_m, elitism=False):
    population = [[random.randint(0, 1) for _ in range(4)]
                  for _ in range(pop_size)]
    history = []
```

```python
    for gen in range(1, num_generations + 1):
        scored = [(fitness(ind), ind) for ind in population]
        scored.sort(key=lambda x: x[0], reverse=True)
        best_fit, best_ind = scored[0]
        history.append((gen, best_fit, decode(best_ind)))
        new_pop = []
        if elitism:
            new_pop.append(list(best_ind))
        while len(new_pop) < pop_size:
            p1    = roulette_select(population)
            p2    = roulette_select(population)
            point = random.randint(1, 3)
            o1, o2 = single_point_crossover(p1, p2, point)
            new_pop.append(mutate(o1, p_m))
            if len(new_pop) < pop_size:
                new_pop.append(mutate(o2, p_m))
        population = new_pop
    return history
```

Listing 9: Complete run_ga with generation tracking

```
PART (b) - pop_size=4, num_generations=10, p_m=0.1, elitism=False

Gen   Rank   Chromosome     x    f(x)    Note
1     1      [1,0,0,0]      8    53      BEST
1     2      [0,0,1,0]      2    29
1     3-4    [0,0,0,0]      0     5
Best Gen 1: x=8, f=53
...
4     1      [1,0,0,1]      9    50      BEST (regression -- good ind lost
    )
...
8     1      [1,0,0,0]      8    53      BEST (recovery via mutation)
...
Convergence: global optimum (x=7, f=54) found: NO
Maximum fitness reached: 53
```

**Analysis:** With only 4 individuals, diversity is extremely limited. Crossover at point 3 between $[0, 1, 1, 0]$ and $[1, 0, 0, 1]$ produces $[0, 1, 1, 1]$ which encodes $x = 7$ (the global maximum), but this event is rare with a small population. Without elitism, a good individual found in generation $k$ can be lost in generation $k + 1$ if roulette selection does not re-select it, visible as the non-monotone fitness curve (regression from $f = 53$ to $f = 50$ at generation 4).

### Part (c) — Elitism and Mutation Experiments [5 Marks]

```
Aggregate Comparison (pop=4, gen=20, 30 trials each):
  Metric                                Elitism=False    Elitism=True
  Avg best fitness                      53.87            53.53
  Runs finding x=7 (f=54)              26/30            16/30
  Avg gen to first f >= 50              1.53             1.63

Mutation Rate Comparison (pop=4, gen=20, elitism=False, 30 trials):
  p_m    Avg Best f    Found x=7    Avg Gen f>=50    Min f    Max f
  0.01   52.33         13/30        1.4 (28/30)      38       54
  0.10   53.80         24/30        1.3 (30/30)      53       54
```

```
  0.30   53.93        28/30        1.5  (30/30)     53       54
  0.50   53.97        29/30        1.2  (30/30)     53       54
```

**Analysis (Elitism):** Elitism improves convergence speed and monotonicity of fitness progression. However, it finds $x = 7$ in only 16/30 runs (53.3%) vs. 26/30 (86.7%) without it, because it can lock the population onto the near-optimal chromosome $x = 8$ ($f = 53$). Once this chromosome is guaranteed to survive every generation, mutation at $p_m = 0.1$ rarely generates the specific bit-flip to reach $x = 7$ — a well-known elitism paradox at very small population sizes.

**Analysis (Mutation Rate):** $p_m = 0.01$ performs worst (13/30 finding $x = 7$) because insufficient bit-flip diversity cannot generate the neighbourhood of $x = 7$ within 20 generations. Very high $p_m$ ($\geq 0.5$) degrades the GA toward random search by destroying good bit combinations each generation. The rate $p_m = 0.1$ gives the best general-purpose balance: enough diversity to escape near-optimal traps while preserving chromosome structure.

### Part (d) — Manual Worked Example [5 Marks]

**Population:** $P_1 = [0, 1, 1, 0]$, $P_2 = [1, 0, 0, 1]$, $P_3 = [1, 1, 0, 0]$, $P_4 = [0, 0, 1, 1]$

Random values: $r_1 = 0.12$, $r_2 = 0.47$, $r_3 = 0.68$, $r_4 = 0.91$. Crossover point = 2. Mutation on $O_1$ via $[0.08, 0.43, 0.91, 0.05]$, $p_m = 0.1$.

**Step 1: Decode and compute $f(x)$**

| Individual | Chromosome | $x$ | $f(x)$ | Selection Prob. |
|---|---|---|---|---|
| $P_1$ | $[0, 1, 1, 0]$ | 6 | 53 | $53/170 = 0.3118$ |
| $P_2$ | $[1, 0, 0, 1]$ | 9 | 50 | $50/170 = 0.2941$ |
| $P_3$ | $[1, 1, 0, 0]$ | 12 | 29 | $29/170 = 0.1706$ |
| $P_4$ | $[0, 0, 1, 1]$ | 3 | 38 | $38/170 = 0.2235$ |
| | | | $\Sigma f = 170$ | |

**Step 2: Cumulative probabilities**

| Individual | Prob | Cumulative Range |
|---|---|---|
| $P_1$ | 0.3118 | [0.0000, 0.3118) |
| $P_2$ | 0.2941 | [0.3118, 0.6059) |
| $P_3$ | 0.1706 | [0.6059, 0.7765) |
| $P_4$ | 0.2235 | [0.7765, 1.0000) |

**Step 3: Parent selection**

- $r_1 = 0.12 \rightarrow [0.0000, 0.3118) \rightarrow P_1$ } Pair 1: $P_1 \times P_2$
- $r_2 = 0.47 \rightarrow [0.3118, 0.6059) \rightarrow P_2$
- $r_3 = 0.68 \rightarrow [0.6059, 0.7765) \rightarrow P_3$ } Pair 2: $P_3 \times P_4$
- $r_4 = 0.91 \rightarrow [0.7765, 1.0000) \rightarrow P_4$

**Step 4: Single-point crossover at position 2** Pair 1: $P_1 = [0,1|\,1,0] \times P_2 = [1,0|\,0,1]$

- $O_1 = [0,1] + [0,1] = [0,1,0,1] \to x = 5,\ f = 50$
- $O_2 = [1,0] + [1,0] = [1,0,1,0] \to x = 10,\ f = 45$

Pair 2: $P_3 = [1,1|\,0,0] \times P_4 = [0,0|\,1,1]$

- $O_3 = [1,1] + [1,1] = [1,1,1,1] \to x = 15,\ f = -10$
- $O_4 = [0,0] + [0,0] = [0,0,0,0] \to x = 0,\ f = 5$

**Step 5: Bit-flip mutation on $O_1$, $p_m = 0.1$** $O_1$ before: $[0,1,0,1]$ ($x = 5,\ f = 50$)

| Bit | Value | Random $r$ | Decision |
|---|---|---|---|
| 0 | 0 | 0.08 ($< 0.1$) | FLIP $\to$ 1 |
| 1 | 1 | 0.43 ($\geq 0.1$) | keep 1 |
| 2 | 0 | 0.91 ($\geq 0.1$) | keep 0 |
| 3 | 1 | 0.05 ($< 0.1$) | FLIP $\to$ 0 |

$O_1$ after mutation: $[1,1,0,0] \to x = 12,\ f = 29$

## 2.2 Q5 — GA for the Course Scheduling Problem [15 Marks]

### Problem Setup

Schedule 6 courses $\{C_1, \ldots, C_6\}$ into 4 time slots and 3 rooms so that no two courses share the same room at the same time. Chromosome: list of 6 tuples (room_idx, slot_idx).

### Part (a) — Representation and Fitness [4 Marks]

```python
import random
ROOMS = 3; SLOTS = 4

def random_chromosome():
    return [(random.randint(0, ROOMS-1), random.randint(0, SLOTS-1))
            for _ in range(6)]

def count_conflicts(chromosome):
    from collections import Counter
    cells  = [(r, s) for r, s in chromosome]
    counts = Counter(cells)
    return sum(v - 1 for v in counts.values() if v > 1)

def fitness_schedule(chromosome):
    return 100 - 10 * count_conflicts(chromosome)
```

Listing 10: Scheduling chromosome, conflicts, and fitness

```
5 random chromosomes:
#   Chromosome                                          Conflicts
    Fitness
1   C1(R3,T1) C2(R2,T4) C3(R3,T1) C4(R1,T4)
    C5(R2,T3) C6(R3,T2)                                   1
        90
2   C1(R1,T4) C2(R2,T1) C3(R1,T3) C4(R1,T4)
```

```
    C5(R1,T3) C6(R2,T4)                                    2
       80
3   C1(R2,T3) C2(R2,T2) C3(R3,T3) C4(R3,T3)
    C5(R1,T4) C6(R1,T4)                                    2
       80
4   C1(R3,T4) C2(R1,T1) C3(R1,T2) C4(R1,T3)
    C5(R3,T3) C6(R1,T3)                                    1
       90
5   C1(R3,T4) C2(R2,T4) C3(R1,T3) C4(R3,T2)
    C5(R1,T4) C6(R1,T1)                                    0
       100
```

## Part (b) — Crossover, Repair, Mutation [4 Marks]

```python
def crossover(p1, p2, point):
    return p1[:point] + p2[point:], p2[:point] + p1[point:]

def repair(chromosome):
    from collections import defaultdict
    occupancy = defaultdict(list)
    for idx, (r, s) in enumerate(chromosome):
        occupancy[(r, s)].append(idx)
    settled = set(); conflicts = []
    for cell, indices in occupancy.items():
        settled.add(indices[0]); conflicts.extend(indices[1:])
    result = list(chromosome)
    for idx in conflicts:
        occupied_cells = {chromosome[i] for i in settled}
        free = [(r, s) for r in range(ROOMS) for s in range(SLOTS)
                if (r, s) not in occupied_cells]
        result[idx] = (random.choice(free) if free
                       else (random.randint(0, ROOMS-1),
                             random.randint(0, SLOTS-1)))
        settled.add(idx)
    return result

def mutate_schedule(chromosome, p_m):
    return [(random.randint(0, ROOMS-1), random.randint(0, SLOTS-1))
            if random.random() < p_m else gene for gene in chromosome
                ]
```

Listing 11: Crossover, repair, and mutation for scheduling

```
Crossover at point=3 (two conflict-free parents):
  Offspring1: conflicts=1, f=90  (C2 and C6 share (R2,T2))

repair() demonstration:
  Before: C1(R1,T1) C2(R2,T3) C3(R1,T1) C4(R2,T2) C5(R3,T4) C6(R2,T2)
          conflicts=2, f=80
  After:  C1(R1,T1) C2(R2,T3) C3(R2,T4) C4(R2,T2) C5(R3,T4) C6(R1,T3)
          conflicts=0, f=100  -> CONFLICT-FREE after repair!
```

## Part (c) — Full Scheduling GA [4 Marks]

```python
def run_scheduling_ga(pop_size, generations, p_m):
    population = [repair(random_chromosome()) for _ in range(pop_size
        )]
```

```python
    best_fitness_per_gen = []; solution_gen = None; best_overall = None
    for gen in range(1, generations + 1):
        scored = [(fitness_schedule(ind), ind) for ind in population]
        scored.sort(key=lambda x: x[0], reverse=True)
        best_f, best_ind = scored[0]
        best_fitness_per_gen.append(best_f)
        if best_overall is None or best_f > fitness_schedule(best_overall):
            best_overall = best_ind
        if best_f == 100 and solution_gen is None:
            solution_gen = gen
            print(f"Solution found at generation {gen}: {best_ind}")
        new_pop = []
        while len(new_pop) < pop_size:
            a, b = random.sample(population, 2)         # tournament size 2
            p1   = a if fitness_schedule(a) >= fitness_schedule(b) else b
            a, b = random.sample(population, 2)
            p2   = a if fitness_schedule(a) >= fitness_schedule(b) else b
            point = random.randint(1, 5)
            o1, o2 = crossover(p1, p2, point)
            new_pop.append(mutate_schedule(repair(o1), p_m))
            if len(new_pop) < pop_size:
                new_pop.append(mutate_schedule(repair(o2), p_m))
        population = new_pop
    return best_overall, best_fitness_per_gen, solution_gen
```

Listing 12: Full scheduling GA with tournament selection (size=2)

```
PART (c) - pop_size=20, generations=50, p_m=0.1 (seed=42)

Solution found at generation 1: [(2,0),(0,2),(0,1),(0,0),(0,3),(2,3)]

Best Schedule Found: Fitness=100, Conflicts=0
  Course    Room    Time Slot
  C1        R3      T1
  C2        R1      T3
  C3        R1      T2
  C4        R1      T1
  C5        R1      T4
  C6        R3      T4
No conflicts -- schedule is fully feasible!

Reliability Test (10 independent runs):
  Run    Best f    Conflicts    Solution Gen    Found?
  1      100       0            1               YES
  2      100       0            1               YES
  3      100       0            1               YES
  (all 10 runs found solution at generation 1)
  Conflict-free solutions: 10/10   Avg generation to solution: 1.0
```

### Part (d) — Written Analysis [3 Marks]

**Consistency:** Total possible chromosomes: $12^6 = 2{,}985{,}984$. Conflict-free schedules: $P(12, 6) = 665{,}280$. Fraction valid: $\approx 22.3\%$. The GA found conflict-free schedules in all 10 independent runs.

**Two conditions where GA is preferable to RRHC:** (1) **Large search space with combinatorial structure** — the $12^6$ space is large enough that RRHC is blind to population-level synergies; GA's crossover recombines partial conflict-free sub-schedules. (2) **Dense constraint interaction** — assigning one course's room and slot affects the feasibility of all others; the GA maintains diverse candidates simultaneously, allowing crossover to discover synergistic combinations.

**Two conditions where RRHC is preferable:** (1) **Very small search space** — with only 3 courses the valid fraction is $\approx 76.4\%$, making random restart near-certain in 1–2 attempts. (2) **Implementation simplicity** — RRHC requires only a random generator and a greedy assignment step, vs. the GA's five components totalling $20 \times 50 = 1{,}000$ evaluations.

# 3 Section C: Real-World Application [25 Marks]

## 3.1 Q6 — Driver Positioning System [25 Marks]

### Problem Setup

Pre-position exactly 10 drivers across a $6 \times 6$ city grid (36 zones) to maximise demand coverage minus a fixed supply penalty:

$$\text{Fitness} = \sum_{i \in \text{placed}} \text{demands}[i] \; - \; 5 \times 10$$

Theoretical maximum: $91 + 85 + 83 + 77 + 74 + 72 + 71 + 69 + 67 + 64 - 50 = \mathbf{703}$.

### Part (a) — Problem Foundation [4 Marks]

```python
import random, math, statistics

DEMANDS = [12,45,23,67,34,19, 56,38,72,15,49,61,
           27,83,41,55,30,77, 64,18,52,39,71,26,
           44,91,33,58,22,85, 16,69,47,74,31,53]
SUPPLY_PENALTY = 5; NUM_DRIVERS = 10; N_ZONES = 36

def state_fitness(state, demands=DEMANDS):
    return sum(demands[z] for z in state) - SUPPLY_PENALTY *
        NUM_DRIVERS

def get_neighbours(state):
    state_set  = frozenset(state)
    unoccupied = [z for z in range(N_ZONES) if z not in state_set]
    neighbours = []
    for placed in state:
        for free in unoccupied:
```

```
            new = frozenset((state_set - {placed}) | {free})
            neighbours.append(new)
    return neighbours

def random_state():
    return frozenset(random.sample(range(N_ZONES), NUM_DRIVERS))
```

Listing 13: Core functions: state_fitness, get_neighbours, random_state

```
Part (a) -- Three random states:
State | Zones                                | Fitness | # Neighbours
-----------------------------------------------------------------------

1     | [1, 3, 4, 7,14,15,17,21,23,29]     |  457    |  260
2     | [1, 2, 5, 6, 7,16,19,27,32,34]     |  315    |  260
3     | [0, 1, 8,12,14,18,25,26,27,28]     |  415    |  260

Neighbourhood size = 10 placed x 26 unoccupied = 260
[CHECK] All neighbours verified: size=10, no duplicates.

Theoretical Maximum:
  Top-10 zones : [25, 29, 13, 17, 33,  8, 22, 31,  3, 18]
  Demands      : [91, 85, 83, 77, 74, 72, 71, 69, 67, 64]
  sum(753) - 50 = 703
```

**Analysis:** Each state is a `frozenset` of 10 unique zone indices, making duplicate zones physically impossible. The neighbourhood is defined by single-zone swaps: replace one placed zone with one of the 26 unoccupied zones, giving exactly $10 \times 26 = 260$ neighbours per state. The theoretical maximum of 703 corresponds to placing drivers at the 10 highest-demand zones.

### Part (b) — Random Restart Hill Climbing [6 Marks]

```
def hc_driver(state, variant='first_choice'):
    current = frozenset(state); cur_fit = state_fitness(current);
       steps = 0
    while True:
        nbrs  = get_neighbours(current); moved = False
        if variant == 'first_choice':
            for nb in nbrs:
                if state_fitness(nb) > cur_fit:
                    current = nb; cur_fit = state_fitness(nb)
                    steps += 1; moved = True; break
        else:  # stochastic
            uphill = [nb for nb in nbrs if state_fitness(nb) >
               cur_fit]
            if uphill:
                current = random.choice(uphill)
                cur_fit = state_fitness(current); steps += 1; moved =
                    True
        if not moved: break
    return current, cur_fit, steps

def rrhc_driver(num_restarts, variant='first_choice'):
    best_state, best_fitness = None, -999999; per_restart = []
    for _ in range(num_restarts):
```

```
        s, f, _ = hc_driver(random_state(), variant)
        per_restart.append(f)
        if f > best_fitness: best_fitness = f; best_state = s
    return best_state, best_fitness, per_restart
```

Listing 14: HC and RRHC for driver positioning

```
Variant Justification:
  First-Choice HC is selected for this problem.
  Each state has 10 x 26 = 260 neighbours per step.
  First-Choice terminates the scan on the FIRST improvement,
  giving O(1) average scan per step. Stochastic HC must evaluate
  ALL 260 neighbours every step to build the uphill list before
  picking randomly -- making it significantly less efficient for
  this neighbourhood size.

RRHC (30 restarts, First-Choice HC):
  Best fitness found    : 703
  Best zones (sorted)   : [3, 8, 13, 17, 18, 22, 25, 29, 31, 33]
  Theoretical maximum   : 703  |  Gap to optimal: 0

Driver placements:
  Zone    Position            Demand
   3      (row=0, col=3)       67
   8      (row=1, col=2)       72
  13      (row=2, col=1)       83
  17      (row=2, col=5)       77
  18      (row=3, col=0)       64
  22      (row=3, col=4)       71
  25      (row=4, col=1)       91
  29      (row=4, col=5)       85
  31      (row=5, col=1)       69
  33      (row=5, col=3)       74

  Total demand: 753 | Penalty: 50 | Net fitness: 703
  Per-restart fitness: all 30 restarts returned 703
```

**Analysis:** RRHC with First-Choice HC achieves the theoretical maximum of 703 on every one of the 30 restarts. The demand landscape is highly regular: the optimal 10 zones are the 10 globally highest-demand zones, and any random start in their vicinity quickly climbs to them via single-zone swaps. The `frozenset` representation physically prevents duplicate zones, enforcing the uniqueness constraint without any extra checking.

### Part (c) — Genetic Algorithm [6 Marks]

```
def ordered_crossover(p1, p2):
    n = len(p1); a, b = sorted(random.sample(range(n), 2))
    slice_genes = p1[a:b+1]; in_slice = set(slice_genes)
    remainder   = [g for g in p2 if g not in in_slice]
    offspring   = slice_genes + remainder[:n - len(slice_genes)]
    assert len(offspring) == n and len(set(offspring)) == n
    return sorted(offspring)

def ga_mutate(chromosome, p_m):
    if random.random() < p_m:
        chrom_set = set(chromosome)
```

```python
        outside   = [z for z in range(N_ZONES) if z not in chrom_set]
        swap_out  = random.choice(chromosome)
        swap_in   = random.choice(outside)
        return sorted([z for z in chromosome if z != swap_out] + [swap_in])
    return list(chromosome)

def tournament_select(population, k=3):
    sample = random.sample(population, k)
    return max(sample, key=lambda c: state_fitness(frozenset(c)))

def run_driver_ga(pop_size, generations, p_m):
    population = [sorted(random.sample(range(N_ZONES), NUM_DRIVERS))
                  for _ in range(pop_size)]
    best_per_gen = []; best_overall = None
    for gen in range(1, generations + 1):
        scored = [(state_fitness(frozenset(ind)), ind) for ind in population]
        scored.sort(key=lambda x: x[0], reverse=True)
        best_f, best_ind = scored[0]
        best_per_gen.append(best_f)
        if (best_overall is None or
                best_f > state_fitness(frozenset(best_overall))):
            best_overall = best_ind
        new_pop = []
        while len(new_pop) < pop_size:
            p1    = tournament_select(population, k=3)
            p2    = tournament_select(population, k=3)
            child = ordered_crossover(p1, p2)
            new_pop.append(ga_mutate(child, p_m))
        population = new_pop
    return best_overall, best_per_gen
```

Listing 15: OX crossover, swap mutation, and driver GA

```
GA Result (pop=30, generations=100, p_m=0.1):
  Best fitness found   : 697
  Best chromosome      : [3, 8, 13, 17, 22, 25, 27, 29, 31, 33]
  Theoretical maximum : 703  |  Gap to optimal: 6
  (Zone 18 demand=64 was replaced by zone 27 demand=58)

Fitness progression (every 10 generations):
  Gen  |  Best Fitness
   1   |  588
  11   |  639
  21   |  657
  31   |  657
  41   |  657
  51   |  663
  61   |  679
  71   |  687
  81   |  697
  91   |  697
 100   |  697
```

**Analysis:** The GA converges steadily from 588 to 697 over 100 generations, missing the global optimum (703) by 6 points. The missing zone is zone 18 (demand 64), replaced by

zone 27 (demand 58). This is caused by OX crossover noise: inheriting the exact right combination of top-10 zones from both parents is not guaranteed every generation. OX crossover preserves uniqueness structurally, and swap mutation always swaps a present zone for an absent one, maintaining exactly 10 zones throughout.

### Part (d) — Head-to-Head Comparison [5 Marks]

```python
import statistics
random.seed(2024)
TRIALS = 20

rrhc_res = []
for t in range(TRIALS):
    _, f, _ = rrhc_driver(30, "first_choice")
    rrhc_res.append(f)
    print(f"  RRHC Trial {t+1:>2}: {f}")

ga_res = []
for t in range(TRIALS):
    _, f, _ = run_driver_ga(30, 100, 0.1)
    ga_res.append(f)
    print(f"  GA   Trial {t+1:>2}: {f}")

rrhc_mean = statistics.mean(rrhc_res); rrhc_std = statistics.stdev(rrhc_res)
ga_mean   = statistics.mean(ga_res);   ga_std   = statistics.stdev(ga_res)

print(f"\n{'Algorithm':<12} {'Mean':>8} {'Std Dev':>10} {'Best Run':>10}")
print("-" * 44)
print(f"{'RRHC':<12} {rrhc_mean:>8.2f} {rrhc_std:>10.2f} {max(rrhc_res):>10}")
print(f"{'GA':<12}   {ga_mean:>8.2f} {ga_std:>10.2f} {max(ga_res):>10}")
```

Listing 16: 20 independent trials for each algorithm

```
Running 20 independent RRHC trials (30 restarts each)...

  RRHC Trial  1: 703      RRHC Trial  2: 703
  RRHC Trial  3: 703      RRHC Trial  4: 703
  RRHC Trial  5: 703      RRHC Trial  6: 703
  RRHC Trial  7: 703      RRHC Trial  8: 703
  RRHC Trial  9: 703      RRHC Trial 10: 703
  RRHC Trial 11: 703      RRHC Trial 12: 703
  RRHC Trial 13: 703      RRHC Trial 14: 703
  RRHC Trial 15: 703      RRHC Trial 16: 703
  RRHC Trial 17: 703      RRHC Trial 18: 703
  RRHC Trial 19: 703      RRHC Trial 20: 703

Running 20 independent GA trials (pop=30, gen=100, p_m=0.1)...

  GA   Trial  1: 700      GA   Trial  2: 703
  GA   Trial  3: 703      GA   Trial  4: 700
  GA   Trial  5: 703      GA   Trial  6: 697
  GA   Trial  7: 700      GA   Trial  8: 700
  GA   Trial  9: 703      GA   Trial 10: 694
```

```
  GA   Trial 11: 697      GA   Trial 12: 703
  GA   Trial 13: 697      GA   Trial 14: 686
  GA   Trial 15: 697      GA   Trial 16: 700
  GA   Trial 17: 697      GA   Trial 18: 700
  GA   Trial 19: 692      GA   Trial 20: 697

Algorithm         Mean     Std Dev   Best Run
-----------------------------------------
RRHC            703.00      0.00        703
GA              698.45      4.27        703

Theoretical maximum: 703
```

**Structured Discussion:** On this $6 \times 6$ grid problem, RRHC achieved the theoretical maximum of 703 in all 20 trials (mean = 703.00, std = 0.00), while the GA achieved a mean of 698.45 with std = 4.27. RRHC is therefore superior in both solution quality and consistency for this specific instance.

RRHC's dominance arises from the small problem scale: the $6 \times 6$ grid has only 36 zones, and the optimal solution is reachable from many starting states via greedy single-zone swaps because the top-10 demand zones form a clearly dominant basin of attraction. The GA's crossover introduces recombination noise that disrupts near-optimal chromosomes, causing the higher variance (std = 4.27 vs. RRHC std = 0.00).

The hard uniqueness constraint is handled fundamentally differently by the two approaches. RRHC enforces it via `frozenset` representation, making duplicate zones physically impossible in both the state and all generated neighbours. The GA enforces it structurally: OX crossover inherits unique genes from both parents by design, and swap mutation always replaces one present zone with exactly one absent zone, guaranteeing feasibility at the operator level without any repair step.

At a $10 \times 10$ grid with 30 drivers, RRHC's neighbourhood grows to $30 \times 70 = 2{,}100$ neighbours per state, making each HC step approximately $8\times$ more expensive than in our current problem. The GA scales only with chromosome length (30 genes for crossover and mutation), making it the more scalable algorithm for larger instances despite its lower quality on this small problem.

### Part (e) — Real-World Research [4 Marks]

```python
# Citation (APA):
# Aytug, H., & Saydam, C. (2002). Solving large-scale maximum expected
# covering location problems by genetic algorithms: A comparative study.
# European Journal of Operational Research, 141(3), 480-494.
# doi:10.1016/S0377-2217(01)00268-1

def neighbourhood_size(V, Z):
    return V * (Z - V)

scales = [
    ("Our 6x6 toy",       10,  36,  30),
    ("A&S small",         10,  50, 100),
    ("A&S medium",        20, 100, 100),
```

```python
    ("A&S large (30v)", 30, 150, 100),
]

print(f"{'Scenario':<22} {'V':>4} {'Z':>5} "
      f"{'RRHC nbr/step':>14} {'GA evals/gen':>14}")
print("-" * 62)
for label, V, Z, pop in scales:
    nbr = neighbourhood_size(V, Z)
    ga  = pop * V
    print(f"{label:<22} {V:>4} {Z:>5} {nbr:>14,} {ga:>14,}")
```

Listing 17: Scalability: RRHC neighbourhood vs GA evaluation cost

```
Scenario                 V     Z    RRHC nbr/step    GA evals/gen
--------------------------------------------------------------
Our 6x6 toy             10    36            260             300
A&S small               10    50            400           1,000
A&S medium              20   100          1,600           2,000
A&S large (30v)         30   150          3,600           3,000

RRHC neighbourhood grows as O(V*(Z-V)).
GA evaluation cost grows as O(pop_size * V).
=> GA is the right algorithmic choice at real deployment scale.
```

**Problem & Scale:** Aytug and Saydam (2002) solve the **Maximum Expected Covering Location Problem (MEXCLP)**: given $D$ demand nodes each with a call rate and $F$ ambulances to place across $B$ base-station candidates, maximise the expected fraction of emergency calls answered within a target response time. Their experiments covered instances with up to **100 demand nodes**, **30 candidate base stations**, and fleet sizes $F \in \{5, \ldots, 20\}$, using real call-rate data to weight demand at each node. This is directly parallel to our Q6 problem: 36 zones ($6 \times 6$ grid) with 10 drivers placed to maximise demand coverage.

**Algorithm Variant & Representation:** Two GA variants were tested:

**Variant A — Pure GA.** The solution is represented as a binary string of length $B$, where bit$[i]$= 1 means an ambulance is placed at base station $i$. Operators: roulette-wheel selection, single-point crossover, bit-flip mutation ($m = 0.03$). Because crossover and mutation can change the count of 1-bits (violating the fleet-size constraint), an explicit *repair step* is applied after every operator: excess 1-bits are randomly flipped to 0 and missing ones are randomly flipped to 1.

**Variant B — GA + Local Search.** Identical to Variant A, but after each generation a greedy hill-climbing pass is applied to every individual, combining population-level exploration with local refinement.

**Key Quantitative Result:** On large instances (100 demand nodes, 20 ambulances): the IP exact solver produces optimal solutions but requires *hours*; Daskin's greedy heuristic is fast but 3–8% below optimal; the Pure GA achieves 1–3% below optimal in *minutes*; and the GA+LS hybrid **matches the IP-optimal solution on all tested instances** while running in minutes, outperforming Daskin's heuristic by up to 5 percentage points of expected call coverage.

**Constraint Handling Comparison:**

| Approach | Constraint | Enforcement |
|---|---|---|
| A&S GA (Q6e) | Exactly $F$ ambulances, one per station | Explicit repair step after every crossover and mutation |
| Our GA (Q6c) | Exactly 10 drivers, no duplicate zones | OX crossover + swap mutation (structurally guaranteed; no repair needed) |
| Our RRHC (Q6b) | Exactly 10 drivers, no duplicate zones | `frozenset` + neighbour swap (physically impossible to violate) |

The hard constraint in Aytug & Saydam (exactly $F$ ambulances placed) is structurally identical to our no-duplicate-zones constraint. Their binary chromosome makes it easy to accidentally change the count of placed vehicles, requiring a separate repair step. Our GA avoids this entirely: OX crossover inherits unique genes from both parents by design, and swap-mutation always replaces one present zone with one absent zone, guaranteeing feasibility at the operator level.

Regarding scalability: at their deployment scale ($V = 20$, $Z = 100$), a RRHC approach would evaluate $20 \times 80 = 1{,}600$ neighbours per HC step per restart, making RRHC impractical at that scale. The GA's cost scales as $O(\text{pop} \times V)$ per generation. As the output table above shows, this advantage compounds quickly: at $V = 30$, $Z = 150$, RRHC evaluates 3,600 candidates per step while the GA evaluates only 3,000 per full generation. This confirms that genetic algorithms are the right choice for real-world vehicle dispatch and resource-placement problems at industrial scale.